Tên: Lê Thiên Hạo

MSSV: 089206010393

K-Means Clustering — cài đặt thuật toán từ đầu

Notebook này cài đặt thuật toán **K-Means** bằng tay (không dùng `sklearn.cluster.KMeans`) để hiểu rõ bên trong thuật toán hoạt động ra sao.

Ý tưởng chung gồm 4 bước, lặp lại nhiều lần:

1. **Khởi tạo**: chọn ngẫu nhiên `k` centroid (tâm cụm).
2. **Assignment**: gán mỗi điểm dữ liệu vào centroid gần nó nhất.
3. **Update**: dời centroid đến vị trí trung bình của các điểm vừa được gán vào nó.
4. **Lặp lại** bước 2–3 cho đến khi không còn điểm nào đổi cụm nữa (hội tụ).

Bước 1: Tạo dữ liệu mẫu và khởi tạo centroid ngẫu nhiên

- Tạo một `DataFrame` gồm 19 điểm dữ liệu 2 chiều (`x`, `y`).
- `k = 3` → muốn phân thành 3 cụm, nên khởi tạo 3 centroid với tọa độ ngẫu nhiên trong khoảng (0, 80), trùng khoảng với dữ liệu để centroid rơi vào đúng vùng có điểm.
- `np.random.seed(200)` giúp kết quả ngẫu nhiên này **lặp lại giống nhau** mỗi lần chạy, dễ so sánh/debug.
- Vẽ scatter plot: các điểm dữ liệu tô màu đen, 3 centroid tô màu đỏ/xanh lá/xanh dương theo `colmap` để dễ theo dõi.

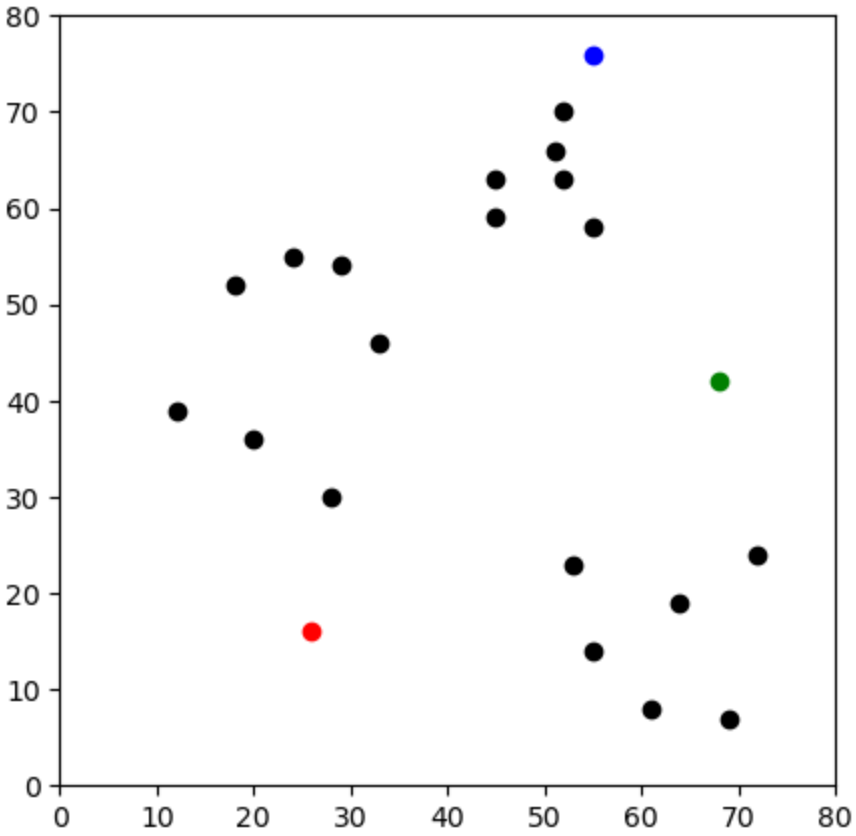
In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'x': [12, 20, 28, 18, 29, 33, 24, 45, 45, 52, 51, 52, 55, 53, 55, 61, 64
    'y': [39, 36, 30, 52, 54, 46, 55, 59, 63, 70, 66, 63, 58, 23, 14, 8, 19,
})

np.random.seed(200)
k = 3
centroids = {
    i + 1: [np.random.randint(0, 80), np.random.randint(0, 80)]
    for i in range(k)
```

```
}

fig = plt.figure(figsize=(5, 5))
plt.scatter(df['x'], df['y'], color = 'k')
colmap = {1: 'r', 2: 'g', 3: 'b'}
for i in centroids.keys():
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
plt.show()
```



Bước 2: Assignment — gán mỗi điểm vào centroid gần nhất

Hàm `assignment(df, centroids)`:

- Với mỗi centroid `i`, tính khoảng cách Euclid từ **mọi điểm** đến centroid đó, lưu vào cột `distance_from_i`.
- `idxmin(axis=1)` trên các cột khoảng cách → tìm centroid gần nhất cho từng dòng (kết quả là tên cột, ví dụ `"distance_from_2"`).
- Cắt bỏ tiền tố `"distance_from_"` để lấy lại số cụm (`closest`), rồi map sang màu tương ứng để vẽ.

`df.head(20)` in ra để kiểm tra khoảng cách và cụm được gán cho từng điểm. Biểu đồ sau đó tô màu mỗi điểm theo centroid gần nó nhất.

In [2]:
```
def assignment(df, centroids):
    for i in centroids.keys():
        df[f'distance_from_{i}'] = (
            np.sqrt(
                (df['x'] - centroids[i][0]) ** 2
                + (df['y'] - centroids[i][1]) ** 2
            )
        )
    centroids_distance_cols = [f'distance_from_{i}' for i in centroids.keys(
    df['closest'] = df.loc[:, centroids_distance_cols].idxmin(axis=1)
    df['closest'] = df['closest'].map(lambda x: int(x.lstrip('distance_from_
    df['color'] = df['closest'].map(lambda x: colmap[x])
    return df

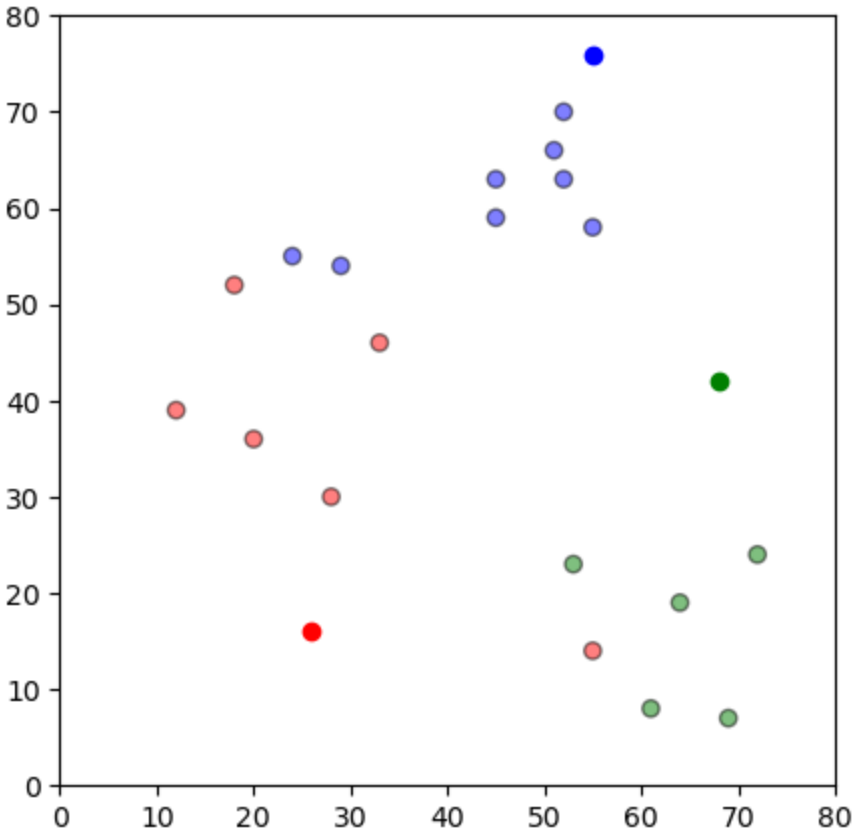

df = assignment(df, centroids)
print(df.head(20))

fig = plt.figure(figsize=(5, 5))
plt.scatter(df['x'], df['y'], color = df['color'], alpha=0.5, edgecolors='k'

for i in centroids.keys():
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
```

	x	y	distance_from_1	distance_from_2	distance_from_3	closest	color
0	12	39	26.925824	56.080300	56.727418	1	r
1	20	36	20.880613	48.373546	53.150729	1	r
2	28	30	14.142136	41.761226	53.338541	1	r
3	18	52	36.878178	50.990195	44.102154	1	r
4	29	54	38.118237	40.804412	34.058773	3	b
5	33	46	30.805844	35.227830	37.202150	1	r
6	24	55	39.051248	45.880279	37.443290	3	b
7	45	59	47.010637	28.600699	19.723083	3	b
8	45	63	50.695167	31.144823	16.401219	3	b
9	52	70	59.933296	32.249031	6.708204	3	b
10	51	66	55.901699	29.410882	10.770330	3	b
11	52	63	53.712196	26.400758	13.341664	3	b
12	55	58	51.039201	20.615528	18.000000	3	b
13	53	23	27.892651	24.207437	53.037722	2	g
14	55	14	29.068884	30.870698	62.000000	1	r
15	61	8	35.902646	34.713110	68.264193	2	g
16	64	19	38.118237	23.345235	57.706152	2	g
17	69	7	43.931765	35.014283	70.405966	2	g
18	72	24	46.690470	18.439089	54.708317	2	g

Out[2]: (0.0, 80.0)



Bước 3: Update — cập nhật lại vị trí centroid

Hàm `update()` dời mỗi centroid đến **vị trí trung bình** (mean toạ độ `x`, `y`) của tất cả các điểm hiện đang thuộc cụm đó. Đây chính là bản chất của K-Means: centroid luôn là trọng tâm của cụm mà nó đại diện.

`old_centroids` được lưu lại bằng `copy.deepcopy` trước khi update, để vẽ **mũi tên** thể hiện centroid đã di chuyển từ vị trí cũ sang vị trí mới (hệ số `0.75` chỉ để mũi tên không che khuất điểm centroid, không ảnh hưởng đến thuật toán).

In [3]:
```
import copy

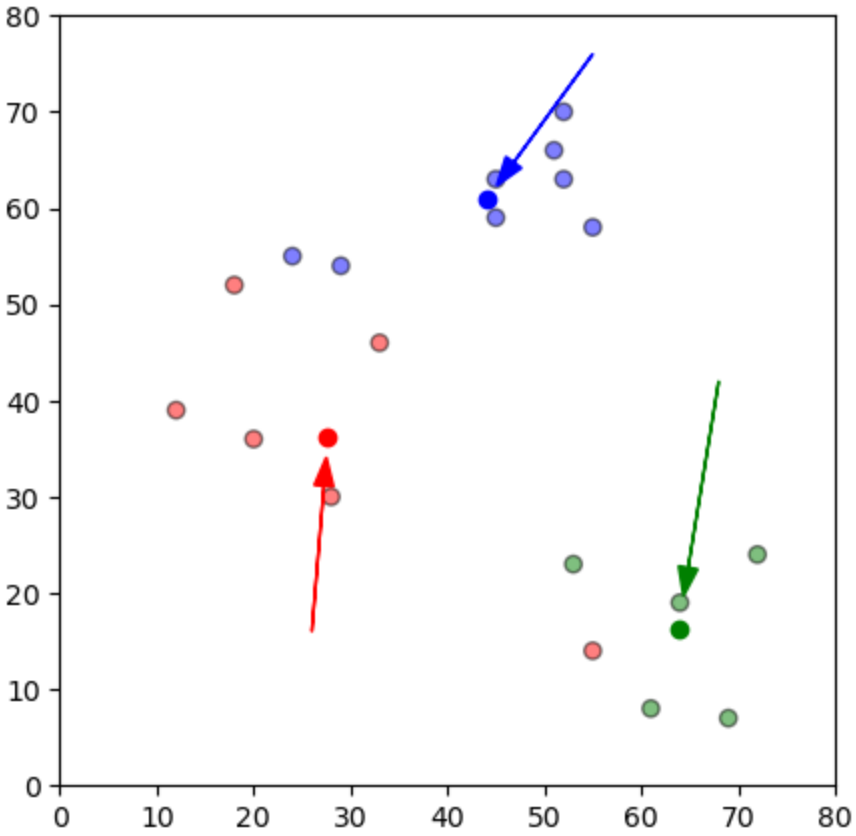
old_centroids = copy.deepcopy(centroids)

def update(k):
    for i in centroids.keys():
        centroids[i][0] = np.mean(df[df['closest'] == i]['x'])
        centroids[i][1] = np.mean(df[df['closest'] == i]['y'])
    return k

centroids = update(centroids)

fig = plt.figure(figsize=(5, 5))
ax = plt.axes()
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
for i in centroids.keys():
```

```
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
for i in old_centroids.keys():
    old_x = old_centroids[i][0]
    old_y = old_centroids[i][1]
    dx = (centroids[i][0] - old_centroids[i][0]) * 0.75
    dy = (centroids[i][1] - old_centroids[i][1]) * 0.75
    ax.arrow(old_x, old_y, dx, dy, head_width=2, head_length=3, fc=colmap[i]
plt.show()
```

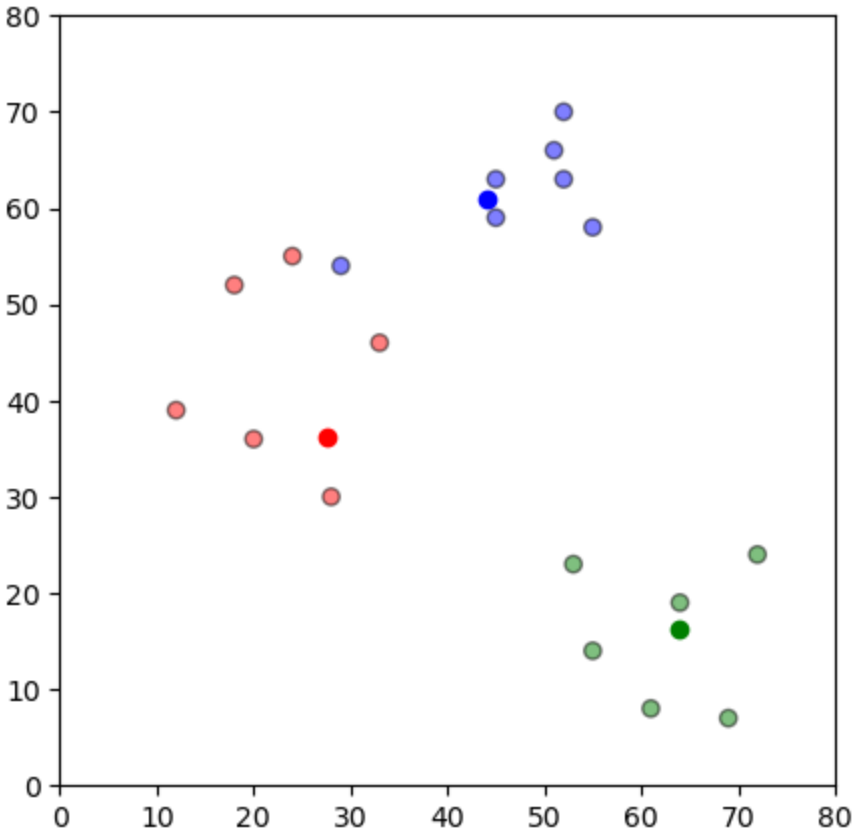


Bước 4: Assignment lại theo centroid mới

Sau khi centroid di chuyển, gọi lại `assignment()` để gán lại các điểm. Vì centroid đã đổi vị trí, một số điểm có thể **đổi màu** (chuyển sang cụm khác) so với biểu đồ ở bước trước.

In [4]:
```
df = assignment(df, centroids)

# Plot results
fig = plt.figure(figsize=(5, 5))
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
for i in centroids.keys():
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
plt.show()
```



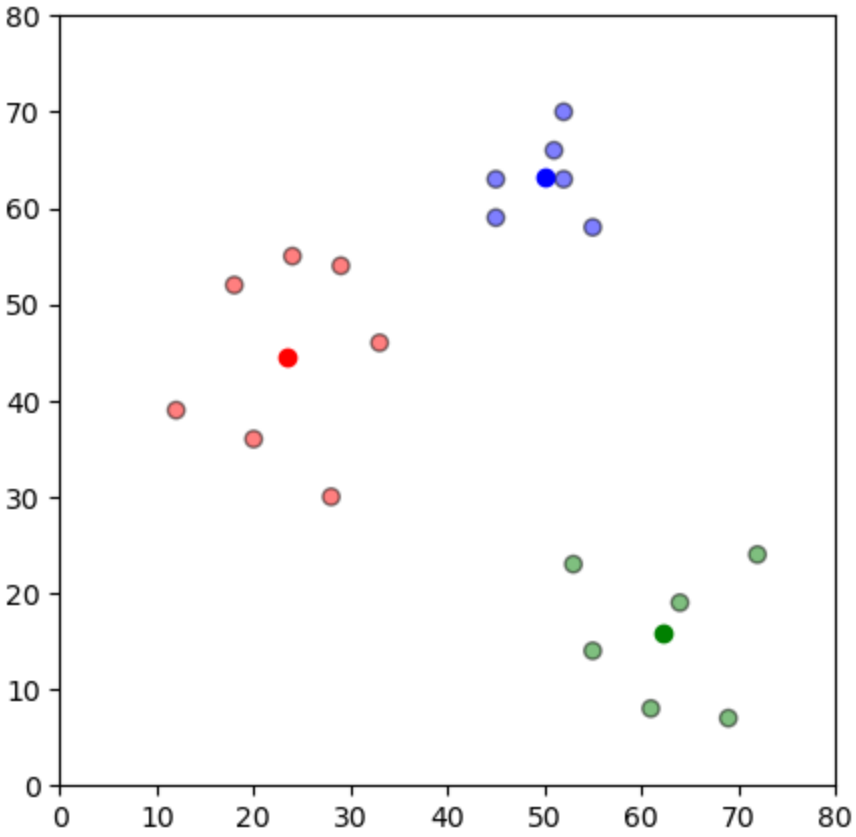
Bước 5: Lặp lại đến khi hội tụ

Tự động hoá việc lặp `update()` → `assignment()` liên tục trong vòng `while True`.

Trước mỗi vòng, `closest_centroids` lưu lại cụm hiện tại của từng điểm. Sau khi update + assignment lại, nếu cột `closest` **không đổi** so với trước đó (`.equals()` trả về `True`) nghĩa là centroid đã ổn định, không điểm nào đổi cụm nữa → thuật toán **hội tụ**, thoát vòng lặp bằng `break`.

In [5]:
```
while True:
    closest_centroids = df['closest'].copy(deep=True)
    centroids = update(centroids)
    df = assignment(df, centroids)
    if closest_centroids.equals(df['closest']):
        break

fig = plt.figure(figsize=(5, 5))
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
for i in centroids.keys():
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
plt.show()
```

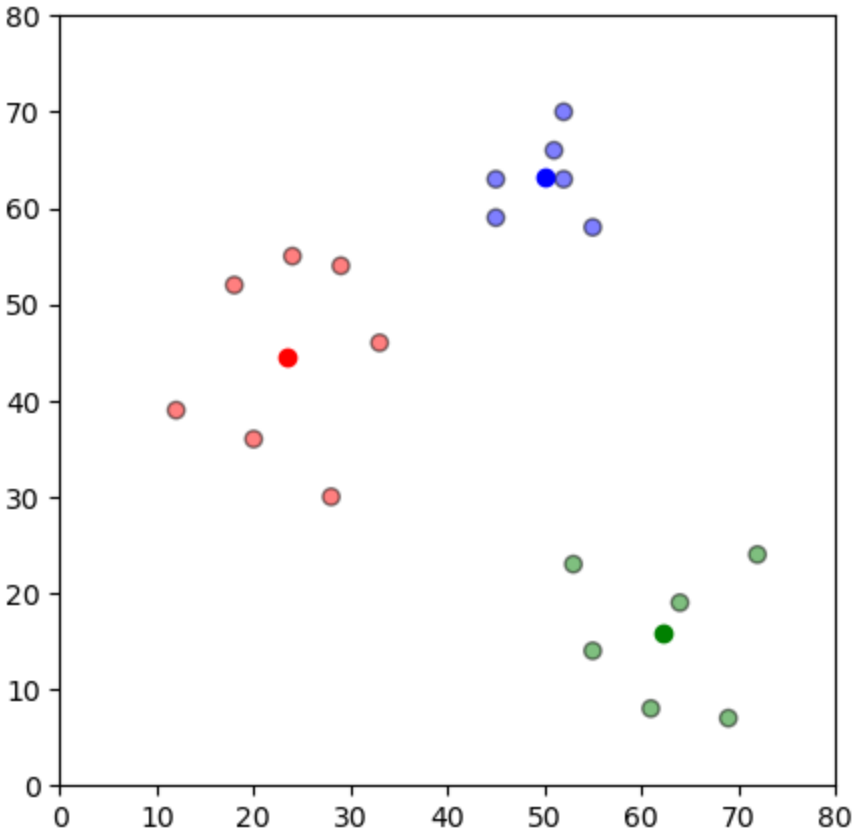


Chạy lại vòng lặp hội tụ

Cell này giống hệt cell phía trên (cùng đoạn code `while True` hội tụ). Vì thuật toán đã hội tụ ở bước trước, chạy lại lần nữa chỉ thực hiện đúng 1 vòng rồi `break` ngay (do `closest` không còn thay đổi) → kết quả và biểu đồ giống hệt cell trước, không có gì thay đổi thêm.

In [6]:
```
while True:
    closest_centroids = df['closest'].copy(deep=True)
    centroids = update(centroids)
    df = assignment(df, centroids)
    if closest_centroids.equals(df['closest']):
        break

fig = plt.figure(figsize=(5, 5))
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
for i in centroids.keys():
    plt.scatter(*centroids[i], color=colmap[i])
plt.xlim(0, 80)
plt.ylim(0, 80)
plt.show()
```



Tóm tắt

Thuật toán K-Means vừa cài đặt thủ công xoay quanh đúng 2 hàm cốt lõi:

- `assignment()` : gán điểm → centroid gần nhất (dựa trên khoảng cách Euclid).
- `update()` : dời centroid → vị trí trung bình các điểm thuộc cụm.

Lặp lại 2 hàm này đến khi việc gán cụm không còn thay đổi chính là toàn bộ "bí mật" đằng sau `sklearn.cluster.KMeans` .